Algorithmen und Datenstrukturen

OOP Programmierung (c#, java)

What is OOP?

Object Oriented Programming

Pros

- Modular
- problems can be broken down into smaller bits
- multiple instances of objects
- data hiding
- eliminating redundant code
- data centered

Cons

- longer
- not always applicable

Compared to Procedural / Functional

Feature	OOP	Procedural
Focus	Objects (data + behavior)	Procedures/functions
Data Handling	Encapsulated within objects	Global or passed between functions
Modularity	High – via classes and objects	Moderate – via functions
Reusability	High – inheritance, polymorphism	Low – copying or re-writing code
Best For	Complex systems, GUI apps, simulations	Simple scripts, linear processes
Code Structure	More structured and hierarchical	More linear and flat

Feature	OOP	Functional
Focus	Objects and classes	Functions and immutability
State	Objects maintain state	Avoids state / uses immutable data
Side Effects	Common and accepted	Avoided (pure functions)
Concurrency	More challenging	Easier due to immutability
Reusability	Through inheritance and composition	Through higher-order functions
Best For	Stateful applications, simulations	Data transformation, concurrency
Learning Curve	Moderate (if procedural background)	Steeper (especially for OOP devs)

Vererbung - Inheritance

Eine Klasse kann von einer anderen erben

- Properties
- Methods

C#

In C# mit `NewClass : BaseClass`

```
// Basisklasse
public class Schiff
{
    public string Name { get; set; }

    public void Fahren()
    {
        Console.WriteLine($"{Name} fährt durch den Kosmos.");
    }
}

// Abgeleitete Klasse
public class Kampfschiff : Schiff
{
    public void Feuern()
    {
        Console.WriteLine($"{Name} feuert mit den Plasma-Kanonen!");
    }
}

// Verwendung
var hootsforce = new Kampfschiff();
hootsforce.Name = "DSS Hootsforce";
hootsforce.Fahren(); // DSS Hootsforce fährt durch den Kosmos.
hootsforce.Feuern(); // DSS Hootsforce feuert mit den Plasma-Kanonen!
```

Java

In Java mit `NewClass extends BaseClass`

```
// Basisklasse
public class Schiff
 {
    public String name;

    public void fahren()
    {
```

```
        System.out.println(name + " fährt durch den Kosmos.");
    }
}

// Abgeleitete Klasse
public class Kampfschiff extends Schiff
{
    public void feuern()
    {
        System.out.println(name + " feuert mit den Plasma-Kanonen!");
    }
}

// Verwendung
public class Main
{
    public static void main(String[] args)
    {
        Kampfschiff hootsforce = new Kampfschiff();
        hootsforce.name = "DSS Hootsforce";
        hootsforce.fahren(); // DSS Hootsforce fährt durch den Kosmos.
        hootsforce.feuern(); // DSS Hootsforce feuert mit den Plasma-Kanonen!
    }
}
```

Interfaces

Interfaces stellen **Schnittstellen** bereit – sie definieren **leere Methoden und Properties**, die eine Klasse **implementieren muss**.
Sie dienen der **Abstraktion** und sorgen für **einheitliches Verhalten** bei verschiedenen Klassen.

C#

```
// Interface
public interface IFlugfaehig
{
    void Fliegen();
    string Antrieb { get; }
}

// Implementierende Klasse
public class Drohne : IFlugfaehig
{
    public string Antrieb => "Rotoren";

    public void Fliegen()
    {
```

```
        Console.WriteLine("Die Drohne hebt ab!");
    }
}

// Verwendung
var drohne = new Drohne();
drohne.Fliegen(); // Die Drohne hebt ab!
Console.WriteLine(drohne.Antrieb); // Rotoren
```

🔧 Java

```
// Interface
public interface IFlugfaehig
 {
    void fliegen();
    String getAntrieb();
}

// Implementierende Klasse
public class Drohne implements IFlugfaehig
{
    public String getAntrieb()
    {
        return "Rotoren";
    }

    public void fliegen()
    {
        System.out.println("Die Drohne hebt ab!");
    }
}

// Verwendung
public class Main
 {
    public static void main(String[] args)
    {
        Drohne drohne = new Drohne();
        drohne.fliegen(); // Die Drohne hebt ab!
        System.out.println(drohne.getAntrieb()); // Rotoren
    }
}
```

Polymorphie

Polymorphie bedeutet, dass **eine Methode mehrere Formen annehmen kann**.
Ein Objekt kann auf unterschiedliche Weise reagieren – abhängig vom **konkreten Typ**, der dahintersteckt.
Das funktioniert oft über **Vererbung** oder **Interfaces**.

C# Beispiel

```
// Basisklasse
public class Schiff
{
    public virtual void Begrüßen()
    {
        Console.WriteLine("Willkommen an Bord.");
    }
}

// Abgeleitete Klasse
public class Piratenschiff : Schiff
{
    public override void Begrüßen()
    {
        Console.WriteLine("Arrr! Willkommen auf dem Piratenschiff!");
    }
}

// Verwendung (Polymorphie)
List<Schiff> flotte = new List<Schiff>
{
    new Schiff(),
    new Piratenschiff()
};

foreach (var schiff in flotte)
{
    schiff.Begrüßen();
}

// Output:
// Willkommen an Bord.
// Arrr! Willkommen auf dem Piratenschiff!
```

Java

```
// Basisklasse
public class Schiff
{
    public void begrüßen()
    {
        System.out.println("Willkommen an Bord.");
    }
}

// Abgeleitete Klasse
public class Piratenschiff extends Schiff
```

```
{
    @Override
    public void begrüßen()
    {
        System.out.println("Arrr! Willkommen auf dem Piratenschiff!");
    }
}

// Verwendung (Polymorphie)
public class Main
{
    public static void main(String[] args)
    {
        Schiff[] flotte = {
            new Schiff(),
            new Piratenschiff()
        };

        for (Schiff schiff : flotte)
        {
            schiff.begrüßen();
        }
    }
}

// Output:
// Willkommen an Bord.
// Arrr! Willkommen auf dem Piratenschiff!
```

Delegates

Ein **Delegate** ist ein **Typ**, der einen Verweis auf eine Methode speichert.
Er erlaubt es, Methoden **wie Variablen** zu behandeln – man kann sie übergeben, speichern und später aufrufen.

C# Beispiel

```
// Delegate definieren
public delegate void MeldungDelegate(string text);

// Methoden, die dem Delegate-Signatur entsprechen
public class Funk
{
    public void Senden(string text)
    {
        Console.WriteLine($"Sende über Funk: {text}");
    }

    public void Rufen(string text)
```

```
    {
        Console.WriteLine($"Rufe laut: {text}");
    }
}

// Verwendung
var funker = new Funk();

MeldungDelegate meldung = funker.Senden;
meldung("Feindliches Schiff voraus!");
// Sende über Funk: Feindliches Schiff voraus!

meldung = funker.Rufen;
meldung("Alle Mann an Deck!");
// Rufe laut: Alle Mann an Deck!
```

Multicast

```
Notifier greetings;

greetings = SayHello;

greetings += SayGoodBye;

greetings("John");

// Hello from John
// Goodbye from John

greetings -= SayHello;

greetings("John");

// Goodbye from John
```

🔧 Java Vergleich (Functional Interface + Lambda)

Java kennt keine echten Delegates, aber man erreicht Ähnliches mit Functional Interfaces und Lambdas:

A **functional interface** in Java is an interface that contains only one abstract method. Functional interfaces can have **multiple default or static methods**, but **only one abstract method**. Runnable, ActionListener, and Comparator are common examples of Java functional interfaces. From Java 8 onwards, lambda expressions and method references can be used to represent the instance of a functional interface.

```
// Functional Interface
@FunctionalInterface
```

```java
interface Meldung
{
    void senden(String text);
}

// Methoden
public class Funk
{
    public void senden(String text)
    {
        System.out.println("Sende über Funk: " + text);
    }

    public void rufen(String text)
    {
        System.out.println("Rufe laut: " + text);
    }
}

// Verwendung
public class Main
{
    public static void main(String[] args)
    {
        Funk funker = new Funk();

        Meldung meldung = funker::senden;
        meldung.senden("Feindliches Schiff voraus!");
        // Sende über Funk: Feindliches Schiff voraus!

        meldung = funker::rufen;
        meldung.senden("Alle Mann an Deck!");
        // Rufe laut: Alle Mann an Deck!
    }
}
```

Other examples

```java
// Define a functional interface
@FunctionalInterface

interface Square
{
    int calculate(int x);
}

class Geeks
{
    public static void main(String args[])
    {
        int a = 5;
```

```
        // lambda expression to
        // define the calculate method
        Square s = (int x) -> x * x;

        // parameter passed and return type must be
        // same as defined in the prototype
        int ans = s.calculate(a);
        System.out.println(ans);
    }
}
```

Datenkapselung (Encapsulation)

Datenkapselung bedeutet, dass die internen Daten eines Objekts (z. B. Felder/Variablen) *privat* gehalten werden und nur über **öffentliche Methoden** (z. B. Getters und Setters) zugänglich sind.
Dadurch schützt man den internen Zustand vor unkontrollierten Zugriffen und kann Validierungen oder Logik in den Zugriffs-Methoden unterbringen.

Vorteile

- **Schutz der Integrität**: Externe Klassen können die privaten Variablen nicht direkt manipulieren.
- **Kontrollierter Zugriff**: Man kann prüfen, ob neue Werte gültig sind oder notwendige Aktionen ausführen.
- **Flexibilität**: Die interne Implementierung kann sich ändern, ohne dass der öffentliche Vertrag (die Methoden/Properties) angepasst werden muss.
- **Lesbarkeit und Wartbarkeit**: Man weiß genau, über welche Schnittstellen Daten verändert oder abgefragt werden.

C# Beispiel

```
using System;

public class EscortShip
{
    // Private field: cannot be accessed directly from outside
    private string name;

    // Public property with getter and setter
    public string Name
    {
        get
        {
            return name;
        }
        set
        {
            if (string.IsNullOrWhiteSpace(value))
```

```
            {
                throw new ArgumentException("Name cannot be empty or
whitespace.");
            }
            name = value;
        }
    }

    // Private field: shield strength points
    private int shieldStrength;

    // Public method to set shield strength with validation
    public void SetShieldStrength(int strength)
    {
        if (strength < 0)
        {
            throw new ArgumentOutOfRangeException(nameof(strength), "Shield
strength must be non-negative.");
        }
        shieldStrength = strength;
    }

    // Public method to get the current shield strength
    public int GetShieldStrength()
    {
        return shieldStrength;
    }

    public void PrintStatus()
    {
        Console.WriteLine($"Ship \"{Name}\" has {shieldStrength} shield points.");
    }
}

// Verwendung
public class Program
{
    public static void Main(string[] args)
    {
        EscortShip ship = new EscortShip();

        // Zugriff über Property (Setter mit Validierung)
        ship.Name = "HMS Blackfang";

        // Zugriff über Methoden (ebenfalls validiert)
        ship.SetShieldStrength(150);

        ship.PrintStatus();
        // Output: Ship "HMS Blackfang" has 150 shield points.

        // Folgender Zugriff wirft eine Exception, da der Name leer ist:
        // ship.Name = "";
    }
```

```
}
```

🔧 Java Beispiel

```
public class EscortShip
{
    // Private field: cannot be accessed directly from outside
    private String name;

    // Private field: shield strength points
    private int shieldStrength;

    // Public getter for name
    public String getName()
    {
        return name;
    }

    // Public setter for name with validation
    public void setName(String name)
    {
        if (name == null || name.trim().isEmpty())
        {
            throw new IllegalArgumentException("Name cannot be empty or
whitespace.");
        }
        this.name = name;
    }

    // Public setter for shieldStrength with validation
    public void setShieldStrength(int strength)
    {
        if (strength < 0)
        {
            throw new IllegalArgumentException("Shield strength must be non-
negative.");
        }

        this.shieldStrength = strength;
    }

    // Public getter for shieldStrength
    public int getShieldStrength()
    {
        return shieldStrength;
    }

    // Helper method to print status
    public void printStatus()
    {
```

```
        System.out.println("Ship \"" + name + "\" has " + shieldStrength + "
shield points.");
    }
}

// Verwendung
public class Main
{
    public static void main(String[] args)
    {
        EscortShip ship = new EscortShip();

        // Zugriff über Setter (mit Validierung)
        ship.setName("HMS Blackfang");
        ship.setShieldStrength(150);

        ship.printStatus();
        // Output: Ship "HMS Blackfang" has 150 shield points.

        // Folgender Aufruf wirft eine IllegalArgumentException, da der Name leer
ist:
        // ship.setName("");
    }
}
```

Structs

Structs are value types that allow you to create small, lightweight objects.
They are typically used for simple data containers and can offer performance benefits because they are stored on the stack (when used locally) rather than the heap.

When to Use Structs

- You need a small data structure to hold related values (e.g., a 2D point, RGB color, or complex number).
- The type will be immutable or have very simple behavior.
- Instances are short-lived and frequently created/destroyed.
- You want to avoid the overhead of heap allocation for many small objects.

C# Example

```
using System;

// Define a struct to represent a 2D point
public struct Point2D
{
    // Public fields (or you can use properties)
    public int X { get; }
```

```
    public int Y { get; }

    // Constructor to initialize fields
    public Point2D(int x, int y)
    {
        X = x;
        Y = y;
    }

    // Method to calculate distance from origin
    public double DistanceFromOrigin()
    {
        return Math.Sqrt(X * X + Y * Y);
    }

    // Override ToString for easy printing
    public override string ToString()
    {
        return $"({X}, {Y})";
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        // Create two Point2D structs
        Point2D p1 = new Point2D(3, 4);
        Point2D p2 = new Point2D(0, 0);

        Console.WriteLine($"Point p1: {p1}");
        // Output: Point p1: (3, 4)

        Console.WriteLine($"Distance of p1 from origin:
{p1.DistanceFromOrigin()}");
        // Output: Distance of p1 from origin: 5

        Console.WriteLine($"Point p2: {p2}");
        // Output: Point p2: (0, 0)
    }
}
```

Notes:

- Structs in C# are declared with the struct keyword.
- Fields or automatic properties can be included, but they should be simple.
- You can define methods, constructors, and even override methods like ToString().
- Structs are value types, so assigning one struct to another copies all fields (no shared reference).

Java

Java doesn't have any structs

```java
// A record automatically creates private final fields, constructor, getters,
// equals(), hashCode(), and toString() for you.
public record Point2D(int x, int y) {
    // You can add additional methods if needed
    public double distanceFromOrigin() {
        return Math.sqrt(x * x + y * y);
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Point2D p1 = new Point2D(3, 4);
        Point2D p2 = new Point2D(0, 0);

        System.out.println("Point p1: " + p1);
        // Output: Point p1: Point2D[x=3, y=4]

        System.out.println("Distance of p1 from origin: " +
p1.distanceFromOrigin());
        // Output: Distance of p1 from origin: 5.0

        System.out.println("Point p2: " + p2);
        // Output: Point p2: Point2D[x=0, y=0]
    }
}
```

Abstract Classes

An **abstract class** is a base class that cannot be instantiated directly. It can contain **abstract methods** (methods without implementation) that **derived classes must override**, as well as regular (concrete) methods or properties. Abstract classes are used to define a common interface and partial behavior for a group of related classes.

When to Use Abstract Classes

- You want to share code (fields, properties, or concrete methods) among subclasses.
- You need to force subclasses to implement certain methods.
- You don't want the base class to be instantiated on its own.

C# Example

```java
using System;

// Abstract base class
public abstract class SpaceVessel
```

```
{
    // Concrete property
    public string Name { get; set; }

    // Abstract method: no implementation here
    public abstract void EngageDrive();

    // Concrete method: shared among all subclasses
    public void ReportStatus()
    {
        Console.WriteLine($"{Name} reports: All systems are nominal.");
    }
}

// Derived class must implement the abstract method
public class Explorer : SpaceVessel
{
    public override void EngageDrive()
    {
        Console.WriteLine($"{Name} engages exploration drive into the abyss!");
    }
}

// Usage
public class Program
{
    public static void Main(string[] args)
    {
        // SpaceVessel vessel = new SpaceVessel(); // Error: Cannot create an
instance of the abstract class

        Explorer hootsforce = new Explorer();
        hootsforce.Name = "DSS Hootsforce";

        hootsforce.ReportStatus();
        // Output: DSS Hootsforce reports: All systems are nominal.

        hootsforce.EngageDrive();
        // Output: DSS Hootsforce engages exploration drive into the abyss!
    }
}
```

🔧 Java Example

```
// Abstract base class
public abstract class SpaceVessel {
    // Concrete field
    private String name;

    public SpaceVessel(String name) {
        this.name = name;
```

```
    }

    // Abstract method: no implementation here
    public abstract void engageDrive();

    // Concrete method: shared among all subclasses
    public void reportStatus() {
        System.out.println(name + " reports: All systems are nominal.");
    }

    // Getter for name (protected so subclasses can use it)
    protected String getName() {
        return name;
    }
}

// Derived class must implement the abstract method
public class Explorer extends SpaceVessel {
    public Explorer(String name) {
        super(name);
    }

    @Override
    public void engageDrive() {
        System.out.println(getName() + " engages exploration drive into the
abyss!");
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        // SpaceVessel vessel = new SpaceVessel("X"); // Error: Cannot instantiate
abstract class

        Explorer hootsforce = new Explorer("DSS Hootsforce");

        hootsforce.reportStatus();
        // Output: DSS Hootsforce reports: All systems are nominal.

        hootsforce.engageDrive();
        // Output: DSS Hootsforce engages exploration drive into the abyss!
    }
}
```

Key Points:

- You cannot create an instance of an abstract class directly.
- Abstract classes may contain both abstract members (to be implemented by subclasses) and concrete members (shared code).

- Subclasses must override all abstract methods, or themselves become abstract.

Exception Handling

Exception handling allows a program to respond to unexpected events or errors (exceptions) in a controlled way, preventing crashes and enabling recovery or cleanup.

Key Concepts

- **try**: Block of code to monitor for exceptions.
- **catch**: Block(s) to handle specific exception types.
- **finally**: (Optional) Block that always executes for cleanup, regardless of whether an exception occurred.
- **throw**: Keyword to raise an exception explicitly.
- **custom exceptions**: User-defined exception classes for domain-specific error conditions.

C# Example

```
using System;

// Custom exception class
public class InsufficientCreditsException : Exception
{
    public InsufficientCreditsException(string message) : base(message) { }
}

public class CreditAccount
{
    private int credits;

    public CreditAccount(int initialCredits)
    {
        credits = initialCredits;
    }

    public void SpendCredits(int amount)
    {
        if (amount > credits)
        {
            // Throw a custom exception if insufficient credits
            throw new InsufficientCreditsException($"Attempted to spend {amount}
credits, but only {credits} available.");
        }
        credits -= amount;
        Console.WriteLine($"Spent {amount} credits, {credits} remaining.");
    }

    public void AddCredits(int amount)
    {
        if (amount < 0)
```

```
        {
            // Throw a built-in exception for invalid argument
            throw new ArgumentOutOfRangeException(nameof(amount), "Cannot add a
negative amount of credits.");
        }
        credits += amount;
        Console.WriteLine($"Added {amount} credits, now {credits} total.");
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        var account = new CreditAccount(50);

        try
        {
            account.SpendCredits(30);
            // Output: Spent 30 credits, 20 remaining.

            account.SpendCredits(25);
            // This line will throw InsufficientCreditsException
        }
        catch (InsufficientCreditsException ex)
        {
            Console.WriteLine($"Credit error: {ex.Message}");
            // Output: Credit error: Attempted to spend 25 credits, but only 20
available.
        }
        catch (Exception ex)
        {
            // General catch for any other exceptions
            Console.WriteLine($"Unexpected error: {ex.Message}");
        }
        finally
        {
            Console.WriteLine("End of credit transaction.");
            // Output: End of credit transaction.
        }

        try
        {
            account.AddCredits(-10);
            // This line will throw ArgumentOutOfRangeException
        }
        catch (ArgumentOutOfRangeException ex)
        {
            Console.WriteLine($"Invalid input: {ex.Message}");
            // Output: Invalid input: Cannot add a negative amount of credits.
        }
    }
}
```

🔧 Java Example

```
// Custom exception class
public class InsufficientCreditsException extends Exception
{
    public InsufficientCreditsException(String message)
    {
        super(message);
    }
}

public class CreditAccount
{
    private int credits;

    public CreditAccount(int initialCredits)
    {
        this.credits = initialCredits;
    }

    public void spendCredits(int amount) throws InsufficientCreditsException {
        if (amount > credits)
        {
            // Throw a custom exception if insufficient credits
            throw new InsufficientCreditsException(
                "Attempted to spend " + amount + " credits, but only " + credits +
" available."
            );
        }
        credits -= amount;
        System.out.println("Spent " + amount + " credits, " + credits + "
remaining.");
    }

    public void addCredits(int amount)
    {
        if (amount < 0)
         {
            // Throw a built-in exception for invalid argument
            throw new IllegalArgumentException("Cannot add a negative amount of
credits.");
        }
        credits += amount;
        System.out.println("Added " + amount + " credits, now " + credits + "
total.");
    }
}

// Usage
public class Main
 {
```

```
    public static void main(String[] args)
    {
        CreditAccount account = new CreditAccount(50);

        try
         {
            account.spendCredits(30);
            // Output: Spent 30 credits, 20 remaining.

            account.spendCredits(25);
            // This line will throw InsufficientCreditsException
        }
        catch (InsufficientCreditsException ex)
        {
            System.out.println("Credit error: " + ex.getMessage());
            // Output: Credit error: Attempted to spend 25 credits, but only 20
available.
        }
        catch (Exception ex)
        {
            // General catch for any other exceptions
            System.out.println("Unexpected error: " + ex.getMessage());
        }
        finally
        {
            System.out.println("End of credit transaction.");
            // Output: End of credit transaction.
        }

        try
        {
            account.addCredits(-10);
            // This line will throw IllegalArgumentException
        }
        catch (IllegalArgumentException ex)
        {
            System.out.println("Invalid input: " + ex.getMessage());
            // Output: Invalid input: Cannot add a negative amount of credits.
        }
    }
}
```

Generics

Generics allow you to write classes, methods, and data structures that work with any data type, without sacrificing type safety.
Sie bieten Flexibilität und Wiederverwendbarkeit, indem sie Typen erst zur Laufzeit festlegen – ganz ohne Casts und Box-/Unboxing-Kram.

Vorteile

- **Typensicherheit**: Fehler werden früh vom Compiler erkannt, nicht erst zur Laufzeit.
- **Wiederverwendbarkeit**: Ein Code-Template für verschiedene Datentypen.
- **Performance**: Kein Box-/Unboxing (in C#) und weniger Casts.
- **Konsistenz**: Einheitliche API für alle Typen, die die gleichen Operationen unterstützen.

C# Example

```
using System;
using System.Collections.Generic;

// Generic class: A simple Pair container
public class Pair<TFirst, TSecond>
{
    public TFirst First { get; }
    public TSecond Second { get; }

    public Pair(TFirst first, TSecond second)
    {
        First = first;
        Second = second;
    }

    public override string ToString()
    {
        return $"({First}, {Second})";
    }
}

public class Program
{
    // Generic method: Swaps two elements in an array
    public static void SwapElements<T>(T[] array, int indexA, int indexB)
    {
        T temp = array[indexA];
        array[indexA] = array[indexB];
        array[indexB] = temp;
    }

    public static void Main(string[] args)
    {
        // Using the generic Pair with different type combinations
        var nameAgePair = new Pair<string, int>("Zoe", 28);
        var coordPair = new Pair<int, int>(3, 4);

        Console.WriteLine(nameAgePair);
        // Output: (Zoe, 28)

        Console.WriteLine(coordPair);
        // Output: (3, 4)

        // Using the generic method SwapElements
```

```
        int[] numbers = { 1, 2, 3, 4 };
        Console.WriteLine($"Before swap: {string.Join(", ", numbers)}");
        // Output: Before swap: 1, 2, 3, 4

        SwapElements(numbers, 1, 3);
        Console.WriteLine($"After swap: {string.Join(", ", numbers)}");
        // Output: After swap: 1, 4, 3, 2
    }
}
```

🔧 Java Example

```
import java.util.Arrays;

// Generic class: A simple Pair container
public class Pair<TFirst, TSecond> {
    private final TFirst first;
    private final TSecond second;

    public Pair(TFirst first, TSecond second) {
        this.first = first;
        this.second = second;
    }

    public TFirst getFirst() {
        return first;
    }

    public TSecond getSecond() {
        return second;
    }

    @Override
    public String toString() {
        return "(" + first + ", " + second + ")";
    }
}

public class Main {
    // Generic method: Swaps two elements in an array
    public static <T> void swapElements(T[] array, int indexA, int indexB) {
        T temp = array[indexA];
        array[indexA] = array[indexB];
        array[indexB] = temp;
    }

    public static void main(String[] args) {
        // Using the generic Pair with different type combinations
        Pair<String, Integer> nameAgePair = new Pair<>("Zoe", 28);
        Pair<Integer, Integer> coordPair = new Pair<>(3, 4);
```

```
        System.out.println(nameAgePair);
        // Output: (Zoe, 28)

        System.out.println(coordPair);
        // Output: (3, 4)

        // Using the generic method swapElements
        Integer[] numbers = { 1, 2, 3, 4 };
        System.out.println("Before swap: " + Arrays.toString(numbers));
        // Output: Before swap: [1, 2, 3, 4]

        swapElements(numbers, 1, 3);
        System.out.println("After swap: " + Arrays.toString(numbers));
        // Output: After swap: [1, 4, 3, 2]
    }
}
```

OOP Concepts

Object-Oriented Programming (OOP) is built on four main pillars that work together to organize code into modular, reusable components. Here's a quick rundown:

1. **Encapsulation**
    - **Definition**: Bundling data (fields) and methods (functions) that operate on that data into a single unit—an object—and restricting direct access to some of an object's components.
    - **Why it matters**: Protects internal state, enforces validation, hides implementation details.
    - **Key benefit**: You expose only what's necessary (via public methods or properties) and keep the rest private.
2. **Inheritance**
    - **Definition**: A mechanism where a new class (subclass/derived class) inherits properties and methods from an existing class (base class).
    - **Why it matters**: Promotes code reuse, establishes an "is-a" relationship (e.g., `KampfShip is-a Ship`).
    - **Key benefit**: You write shared behavior once in the base class, and subclasses automatically get it (with the option to override).
3. **Polymorphism**
    - **Definition**: The ability for different classes to be treated as instances of the same base type, with method calls resolving to the appropriate implementation at runtime.
    - **Why it matters**: You can write code against an abstract interface or base class and swap in any subclass without changing the calling code.
    - **Key benefit**: Flexible, extensible design—"one method, many behaviors."
4. **Abstraction**

- **Definition**: Hiding complex implementation details behind a simple interface or abstract class, exposing only the essential features.
- **Why it matters**: Reduces complexity for the caller; you only see “what” happens, not “how.”
- **Key benefit**: Simplifies the mental model of your system; you focus on functionality rather than internal mechanics.

Example: Tying Them Together in C#

```
// Abstract base class (Abstraction + Inheritance)
public abstract class SpaceVessel
{
    private int hullIntegrity; // Encapsulated field

    public string Name { get; set; } // Public property

    public SpaceVessel(string name, int initialIntegrity)
    {
        Name = name;
        hullIntegrity = initialIntegrity;
    }

    // Abstract method (Abstraction)—subclasses must implement
    public abstract void EngageDrive();

    public void ReportStatus() // Concrete method
    {
        Console.WriteLine($"{Name} hull integrity: {hullIntegrity}");
    }

    // Encapsulated method to modify internal state
    protected void DamageHull(int amount)
    {
        hullIntegrity = Math.Max(0, hullIntegrity - amount);
    }
}

// Derived class (Inheritance + Polymorphism)
public class PiratenJagdbote : SpaceVessel
{
    public PiratenJagdbote(string name, int initialIntegrity)
        : base(name, initialIntegrity) { }

    public override void EngageDrive() // Polymorphic implementation
    {
        Console.WriteLine($"{Name} tears through the void with abyssal engines!");
        DamageHull(5); // Using an encapsulated method
    }
}

// Usage
public class Program
```

```
{
    public static void Main(string[] args)
    {
        List<SpaceVessel> fleet = new List<SpaceVessel>
        {
            new PiratenJagdbote("DSS Hootsforce", 100),
            new PiratenJagdbote("Firebird", 50)
        };

        foreach (var ship in fleet)
        {
            ship.ReportStatus();   // Calls base logic
            ship.EngageDrive();    // Calls overridden logic (Polymorphism)
            ship.ReportStatus();
            // Output:
            // DSS Hootsforce hull integrity: 100
            // DSS Hootsforce tears through the void with abyssal engines!
            // DSS Hootsforce hull integrity: 95
            // Firebird hull integrity: 50
            // Firebird tears through the void with abyssal engines!
            // Firebird hull integrity: 45
        }
    }
}
```

Clean Code

Clean Code is all about writing code that's **readable**, **maintainable**, and **easy to understand**—even if you come back months later (or your first mate tries to debug at 2 AM). Here are the core principles:

1. **Meaningful Names**
    - Variables, methods, and classes should have descriptive names.
    - Avoid abbreviations or single letters (unless it's a short-lived loop index).
    - Use domain terminology (e.g., `AbyssJump()` instead of `DoAction()` if it specifically jumps to another system).
2. **Single Responsibility Principle (SRP)**
    - Each class or method should do **only one thing** and do it well.
    - If a method tries to load from disk and parse JSON and update UI, split it into three methods.
    - Keeps code easier to test and debug.
3. **Don't Repeat Yourself (DRY)**
    - Duplicate logic is a maintenance nightmare.
    - Extract common code into helper methods or base classes.
    - If "calculate jump range" appears in two places, make a single `CalculateAbyssRange()` method.
4. **Small Functions / Methods**

- Aim for methods that fit on one screen without scrolling.
- Each method does a clear, focused task and has a descriptive name.
- Example:

```
// NOT CLEAN:
public void ProcessOrder(Order order) {
    // 50 lines doing validation, calculation, logging, database
writes, and notification...
}

// BETTER:
public void ProcessOrder(Order order) {
    ValidateOrder(order);
    decimal total = CalculateOrderTotal(order);
    SaveOrderToDatabase(order, total);
    SendConfirmation(order);
}
```

5. **Consistent Formatting**

- Consistent indentation, brace style, and spacing.
- Use your team's style guide (or pick one if you fly solo).
- Example (C# – K&R style):

```
public void JumpToSystem(string systemName)
{
    if (string.IsNullOrEmpty(systemName))
    {
        throw new ArgumentException("System name is required.");
    }

    // …perform jump logic…
}
```

6. **Meaningful Comments (When Needed)**

- Comments should explain **why** something is done, not **what** (the code should already show what).
- Remove outdated comments—stale comments are worse than none.
- Example:

```
// BAD COMMENT: // Increment counter
counter++;

// GOOD COMMENT:
// We increment the counter here to offset the initial index shift from
```

```
the Abyss drive calibration.
counter++;
```

7. **Prefer Immutability When Possible**

   - Immutable objects are less error-prone and thread-safe by default.
   - Use `readonly` fields (C#) or `final` fields (Java) for data that should not change once initialized.
   - Example (C#):

```
public class Coordinates
{
    public readonly int X;
    public readonly int Y;

    public Coordinates(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

8. **Error Handling and Exceptions**

   - Don't swallow exceptions—handle them appropriately or propagate them.
   - Use `try`/`catch`/`finally` sparingly; don't wrap every line in a `try`.
   - Example (C#):

```
try
{
    account.SpendCredits(100);
}
catch (InsufficientCreditsException ex)
{
    // Log the issue, inform the user, and let them retry
    Console.WriteLine($"Not enough credits: {ex.Message}");
}
```

9. **Unit Tests and Testable Code**

   - Design your classes so dependencies can be injected (e.g., via constructor).
   - Keep methods small and focused so they're easy to test.
   - Example (C#):

```
public class CreditAccount
{
    private readonly ILogger _logger;
```

```
    public CreditAccount(int initialCredits, ILogger logger)
    {
        Credits = initialCredits;
        _logger = logger;
    }

    public int Credits { get; private set; }

    public void Spend(int amount)
    {
        if (amount > Credits)
        {
            _logger.Log("Insufficient credits.");
            throw new InvalidOperationException("Not enough credits.");
        }
        Credits -= amount;
    }
}
```

10. **Refactor Often**
    - If you see “code smells” (long methods, duplicated logic, deeply nested conditionals), carve off a tidy little refactoring.
    - Keep your ship tight—routine maintenance prevents catastrophic failures later.

> **Pirate’s Note**:
> “Clean code is like a well-kept deck. If every plank’s in place, you won’t trip over a loose nail when battling foes—or merging code!”
